Spec-Driven Project Structure

A reusable way to build a project with Claude Code, taken from what worked on Cartograph: phases, specs, rules, manual acceptance, automated QA, AI evals and code review. Plus what to add next time.

Goran Novaković · November 2026

1. The idea in one page
2. Folder layout
3. The three documents (with templates)
4. The phase loop
5. Quality layers: who checks what
6. Code review with CodeRabbit
7. QA automation and evals
8. What to add next time
9. New-project checklist

---

1. The idea in one page

Nothing gets built without a written spec. The work is cut into phases, each small enough to check by hand in a browser in ten minutes. Claude builds one phase, checks what a terminal can check, and stops. You run the acceptance check. If it passes, it becomes one commit. If it fails, you reset and fix the spec instead of prompting on top of broken code.

Three documents carry the project, each with a different lifespan:

DOCUMENT	HOLDS	READ	CHANGES
`CLAUDE.md`	Rules only: stack, how we work, how to talk, layout rules, conventions, things not to do	Every prompt (automatically)	Rarely
`docs/project-doc.md`	What the product is and *why* every decision was made	The section you need, when you need it	When a decision changes
`docs/specs/phase-NN.md`	One phase: goal, behaviour, constraints, acceptance check	At the start of that phase	Written just before the phase; frozen once it passes

> **Why it works:** rules stay short because the reasons live elsewhere; specs stay small because the product vision lives elsewhere; and the git log doubles as a progress bar, since every commit is one phase that passed.

2. Folder layout

```
my-project/
├── CLAUDE.md                  # rules, read on every prompt
├── .coderabbit.yaml           # review rules (mirrors CLAUDE.md "things not to do")
├── .env.example               # every variable the app needs, no values
├── .github/workflows/ci.yml   # typecheck · lint · build · signed-out QA on every PR
├── .claude/
│   ├── settings.json          # shared permissions + hooks (committed)
│   └── commands/phase.md      # /phase NN: "read CLAUDE.md + spec, build, stop"
├── docs/
│   ├── project-doc.md         # the what and the why
│   ├── decisions.md           # dated log of decisions changed mid-project
│   └── specs/
│       ├── phase-00.md        # setup: stack, auth, env, CI
│       ├── phase-01.md
│       └── ...
├── e2e/                       # Playwright QA
│   ├── signed-out/
│   └── signed-in/
├── evals/                     # scripts that turn AI quality into a number
└── src code ...               # your choice; CLAUDE.md states behaviour rules, not filenames
```

3. The three documents

3.1 CLAUDE.md — the rules

Short, imperative, no reasoning (that goes in the project doc). Keep these sections; change the content per project.

```
# <Project>
One paragraph: what it is + the single rule everything rests on.
Why these rules exist is in docs/project-doc.md.

## Stack
Framework + versions, libraries, package manager.
"If unsure about an API, read the docs in the installed package."

## How we work
Spec driven. Nothing gets built without a spec.
- docs/project-doc.md — the what and why. Read the part you need.
- docs/specs/phase-NN.md — behaviour + acceptance check, never filenames.
Starting a phase: read this file and that spec. Build it and stop.
Fresh context? Look at docs/specs/ and git log; tell me where we are first.
The acceptance check is mine. Yours: types, lint, build.
When a phase comes out wrong, I reset rather than patch.

## How to talk to me
Short. Ask one question with your recommended answer attached.
Need a key/token? Say exactly what and where, then stop.
Say when something didn't work.

## How the code is laid out
Behaviour rules (e.g. "core logic imports no framework",
"DB access only in server code", "one place builds the AI client").

## Conventions
Strict TS, no any. Comment decisions, not syntax. UI tone.

## Things not to do
Breaking one of these is worse than not finishing.
- Don't install a package without asking.
- Don't build ahead of the current phase.
- Don't leave the build broken. Don't weaken a check to make it pass.
- + the 2–4 product rules that must never break.
```

3.2 project-doc.md — the what and the why

Written once before phase 1. The reasoning matters more than the decision: Claude will hit forks the doc doesn't cover, and needs to know which way the thinking leans.

```
# <Project>
## What it is            — the product in a paragraph, as a user sees it
## The problem           — why it needs to exist now
## Who it's for          — the one person every feature is measured against
## Why this hasn't worked before   — competitors, failures, the bet
## The rule everything rests on    — the non-negotiable, and why
## Scope / Out of scope, and why
## How it works          — pipeline / architecture in prose
## The interface         — layout settled early; later phases add, not rearrange
## Where this is likely to go wrong — the known hard parts
## What has to be true before this ships — concrete, checkable list
```

3.3 phase-NN.md — one phase

Behaviour, never filenames. If you can't write the acceptance check, the phase isn't ready.

```
# Phase NN — <Name>

**Goal.** One or two sentences: what's true after this phase that wasn't before.

## Build
- **Bold lead-in.** What the user can see or do. Behaviour only.

## Constraints
- **Bold rule.** Limits and edge cases: what must stay true, what must NOT happen,
  what happens on failure, what stays unchanged from earlier phases.

## Acceptance check
1. Something you do by hand in a browser, and what you should see.
2. Include at least one failure case (bad input, network off, wrong user).
3. Include one check that a rule held (e.g. "search the core for X: only in adapter").
```

> Good phase size: 1–3 days of work, one commit, an acceptance check of 3–6 steps. Cartograph went 14 phases from auth to re-runs.

4. The phase loop

1 · Spec	2 · Ambiguity pass	3 · Build	4 · Terminal checks	5 · Acceptance	6 · PR + review	7 · Merge
You write phase-NN.md just before starting	**Claude** reads it, asks questions *before* building	**Claude** on a branch `phase-NN`	**Claude** types, lint, build, scripts	**You** by hand, in a browser	**CI + CodeRabbit**	one squash commit: `Phase NN: <name>`

If step 5 fails: don't prompt three more times on top. Decide whether the code or the spec was wrong. If the spec was ambiguous, fix it, reset the branch, rebuild. Small, obvious bugs can be patched; wrong direction gets reset.

After it passes: turn the stable parts of the acceptance check into Playwright tests (section 7), so later phases can't silently break it.

Prompt to start a phase (or save it as `.claude/commands/phase.md` and type `/phase 07`):

```
Read CLAUDE.md and docs/specs/phase-$ARGUMENTS.md.
Before building, list anything in the spec that is ambiguous, each with the
answer you'd pick. Wait for me. Then build only this phase, run typecheck,
lint and build, and tell me in a few lines what to check.
```

5. Quality layers: who checks what

LAYER	WHO	WHEN	CATCHES
TypeScript strict, ESLint, build	Claude, CI	Every change	Broken code, bad types
Script checks (e.g. `npm run parse` on a known repo)	Claude	Before saying done	Core logic wrong, verifiable without a browser
Acceptance check	You, by hand	End of each phase	Does it actually do what the spec says
Playwright QA	CI / `npm run qa`	Every PR after the phase passed	Regressions in earlier phases
AI evals	`npm run eval:*`	After prompt/model changes	AI quality drop, invented facts
CodeRabbit review	Bot on the PR	Every PR	Rule violations, bugs, security smells
Security review	`/security-review`	Before release, after auth/DB phases	Auth gaps, RLS, secrets, injection

> Order matters: the human acceptance check comes *first* and is never replaced by automation. Automated QA comes *after*, to keep what already passed from breaking.

6. Code review with CodeRabbit

Commit the config file so the review rules live in the repo, next to CLAUDE.md. The most valuable part is `path_instructions`: copy your "things not to do" and layout rules into it, so the reviewer enforces the same rules Claude was given.

```
# .coderabbit.yaml
language: en-US
reviews:
  profile: chill            # fewer nitpicks; rules below carry the weight
  request_changes_workflow: false
  high_level_summary: true
  auto_review:
    enabled: true
    base_branches: [main]
  path_instructions:
    - path: "parser/**"
      instructions: |
        Must not import next, react or the database client.
        No framework-specific checks here; those belong in adapters.
    - path: "app/**/*.tsx"
      instructions: |
        No database access inside components; it belongs in server code.
    - path: "supabase/migrations/**"
      instructions: |
        Every new table must enable row-level security with a policy.
    - path: "**/*.ts"
      instructions: |
        No `any`, no unchecked casts. AI client only constructed in lib/ai.
  path_filters:
    - "!**/*.lock"
    - "!lib/database.types.ts"
```

Treat CodeRabbit comments like spec questions: Claude can read them (`gh pr view --comments`) and fix the ones that are real, but you decide. A comment that contradicts CLAUDE.md loses.

7. QA automation and evals

Playwright QA

- Split by auth state: `e2e/signed-out` needs nothing; `e2e/signed-in` reuses a session you save once by hand (`npm run qa:login`). Auth providers' sign-in is yours to complete, not a script's.
- `workers: 1` when tests share one dev server and one database.
- `trace: "retain-on-failure"` so a failure can be opened and replayed.
- Test behaviour from the specs: "a second organization sees none of the first one's rows", not "button is blue".
- Write tests after the phase passes, from its acceptance check. Start this from phase 2, not at the end.

AI evals (only if the app uses AI)

- One script per claim you make about the AI: e.g. "never names a file that doesn't exist" → `eval:paths`.

- Each eval must be able to fail: give it a way to inject a known bad answer (Cartograph's `--splice`) and confirm it's caught.
- Run evals over real traced runs (LangSmith), so a flagged result can be opened and checked by hand.
- Output is a number and a list of failures, never a vibe.

8. What to add next time

Things Cartograph did without, or added late, that would make the next project smoother.

ADD	WHY
Branch + PR per phase	Cartograph committed straight to main. A PR per phase gives CodeRabbit and CI something to run on, and makes "reset the phase" a branch delete.
Phase 00: setup	Stack, auth, env, CI, CodeRabbit, Playwright skeleton before any feature. Then QA grows with each phase instead of arriving at phase 14.
CI workflow	`typecheck · lint · build · qa (signed-out)` on every PR. Claude's "I checked" becomes verifiable.
`npm run check`	One script: `tsc --noEmit && eslint && next build`. One command for Claude and CI.
`.env.example` + real README	The README is still the create-next-app default. List every env var, how to run, how to run QA and evals.
`docs/decisions.md`	A dated line whenever a decision in project-doc changes mid-build, with the reason. Stops the doc and the code drifting apart.
Consistent commit names	Exactly `Phase NN: <name>` for phase commits. Cartograph's log has duplicates and mixed styles, which makes "where are we" harder in a fresh context.
Shared `.claude/settings.json`	Commit allowed commands (npm run check, gh, git status) so permission prompts are the same on every machine. Keep personal ones in settings.local.json.
A Stop hook running typecheck	Claude can't end a turn with broken types without seeing the error. Enforces "don't leave the build broken" mechanically.
`/phase` slash command	The start-a-phase prompt in one word, identical every time (section 4).
Security review at fixed points	Run `/security-review` after the auth phase, after the database phase, and before release, not once at the end.
Template repo	Put CLAUDE.md skeleton, spec template, CI, CodeRabbit config and Playwright config in a GitHub template repo. New project = "Use this template".

9. New-project checklist

1. Create from the template repo.
2. Write `docs/project-doc.md`: what, who, the one rule, scope, out of scope, risks, ship criteria.
3. Fill `CLAUDE.md`: stack, the product's "things not to do", layout rules.
4. Copy the product rules into `.coderabbit.yaml` `path_instructions`.
5. Sketch the phase list (one line each) at the bottom of project-doc. Write only phase 00 in full.
6. Phase 00: setup, CI green, CodeRabbit installed on the repo, one Playwright test passing.
7. For each phase: spec → ambiguity pass → build → check → acceptance → PR → merge → QA tests from the check.
8. Before release: run every "What has to be true before this ships" item, full QA, evals, security review.